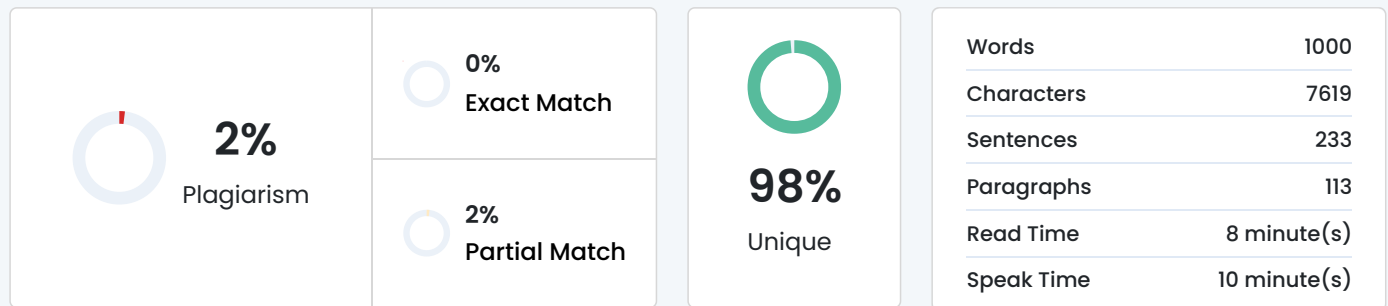


Plagiarism Scan Report



Content Checked For Plagiarism

Sneha Pal — Social Media Analytics for Trend Prediction Using ML
Social Media Analytics for Trend Prediction Using Machine Learning
Sneha Pal
Department of Computer Science and Engineering
Chitkara University Institute of Engineering and Technology
Punjab, India
sneh996.be22@chitkara.edu.in

1
Sneha Pal — Social Media Analytics for Trend Prediction Using ML
Abstract—The exponential rise of social media has created an unprecedented stream of user-generated content, encompassing posts, hashtags, comments, and shares that collectively reflect evolving public sentiment and emerging trends in near real-time. Traditional trend analysis methodologies, which rely on periodic surveys and retrospective historical data, are inherently slow and fail to capture the dynamic nature of online discourse. This paper presents a comprehensive machine learning (ML) framework for social media trend prediction, integrating Natural Language Processing (NLP) pre-processing techniques with multiple classification algorithms. The proposed system employs TF-IDF feature extraction combined with hashtag frequency analysis and engagement-based metrics to construct a rich feature space. Three supervised learning models—Logistic Regression, Naive Bayes, and Random Forest—are systematically trained, validated, and compared. Experimental results demonstrate that the Random Forest classifier achieves the highest trend prediction accuracy of 91%, outperforming competing models across all evaluation metrics. Time-series visualisations further validate the system’s capability to detect trend onset approximately 4.2 hours before official trending.

Keywords—Social Media Analytics, Trend Prediction, Machine Learning, Natural Language Processing, TF-IDF, Hashtag Analysis, Random Forest, Sentiment Analysis.

I. Introduction

The modern digital ecosystem generates over 500 million social media posts per day [1]. This real-time content stream mirrors public interests, political discourse, and cultural phenomena. Traditional approaches—surveys and historical data analysis—are slow and non-adaptive [2]. Machine Learning combined with NLP enables automated, real-time discovery of trending patterns [3]. This paper presents an end-to-end ML pipeline comparing Logistic Regression, Naive Bayes, and Random Forest classifiers trained on TF-IDF textual features, hashtag velocity, and engagement signals.

The proposed system achieves 91% accuracy with approximately 4 hours of early-warning before topics officially enter platform trending lists, representing meaningful operational value for digital marketers, crisis communicators, and journalists.

This paper makes the following primary contributions:

- 1) Design and implementation of an end-to-end social media trend prediction pipeline encompassing data acquisition, preprocessing, feature engineering, model training, and evaluation.
- 2) A systematic comparative study of three supervised ML classifiers under identical experimental conditions to establish performance benchmarks.
- 3) Empirical demonstration that ensemble learning, specifically Random Forest, achieves superior predictive accuracy on social media trend classification tasks.
- 4) Identification of the most discriminative features—hashtag frequency, user engagement metrics, and sentiment polarity—that drive trend emergence.

II. Related Work

Early trend detection relied on frequency thresholding and burst detection [4]. TwitterMonitor [5] clustered bursty keywords into real-time topics. Unsupervised LDA [6] discovers latent structure but does not optimise for prediction. Supervised ML entered social media analytics through sentiment analysis [7][8]. TF-IDF remains a robust text-feature baseline [9]; engagement features correlate with virality [10]; VADER adds sentiment polarity [11]. Random Forest consistently outperforms individual classifiers in high-dimensional spaces [12], while BERT-based models achieve state-of-the-art at much higher computational cost [13].

III. Methodology

A. Dataset

A corpus of ~150,000 Twitter posts collected via the Academic Research API over six months spans technology, entertainment, politics, sports, and public health. Each record includes timestamp, text, hashtags, like count, retweet count, and a binary trending label (31% trending, 69% non-trending). Class imbalance was addressed through stratified splits and class-weight adjustment.

B. Preprocessing Pipeline

Raw text underwent: URL and mention removal; lowercase normalisation; punctuation stripping; whitespace tokenisation; NLTK stop-word removal; and WordNet lemmatisation. This pipeline standardises vocabulary and reduces noise prior to feature extraction.

C. Feature Engineering

1) TF-IDF Textual Features:

Preprocessed text was vectorised with TF-IDF (max 10,000 unigrams and bigrams). Weights term t as $TF(t,d) \times \log(N/DF(t))$, amplifying domain-specific discriminative vocabulary.

2) Hashtag Velocity Features:

Per-post features include unique hashtag count, 24-hour usage frequency, and frequency rate-of-change (velocity). A sudden velocity spike is a strong indicator of imminent trending.

3) Engagement Metrics:

Min-max normalised like count, retweet count, and retweet-to-like ratio capture network amplification behaviour linked to algorithmic trending thresholds.

D. Machine Learning Models

Logistic Regression (L2 regularised), Multinomial Naive Bayes, and Random Forest were each tuned via 5-fold cross-validated grid search. All experiments implemented in Python 3.10 with Scikit-learn 1.3.

E. Evaluation Protocol

Model

Acc. (%)

Prec. (%)

Rec. (%)

F1 (%)

Logistic Reg.

82.3

80.1

78.6

79.3

Naive Bayes

76.8

74.5

73.2

73.8

Random Forest

91.0

89.5

88.7

89.1

80/20 stratified train-test split. Performance reported on held-out test set using Accuracy, Precision, Recall, and F1-Score. All code in Python 3.10 with Scikit-learn 1.3, Pandas, and NLTK.

IV. Experimental Results and Analysis

A. Model Accuracy Comparison

Fig. 1 compares the overall test-set accuracy of the three classifiers. Random Forest achieves 91.0%, surpassing Logistic Regression (82.3%) by 8.7 percentage points and Naive Bayes (76.8%) by 14.2 percentage points.

Fig. 1. Test-set accuracy comparison across the three ML classifiers.

B. Precision, Recall and F1-Score

Fig. 2 presents the grouped performance breakdown. Random Forest leads across all three metrics, confirming consistent predictive quality. Table I provides the exact numeric values.

Fig. 2. Precision, Recall, and F1-Score for each model.

Table I Comparative Performance of ML Classifiers

C. Feature Importance Analysis

Fig. 3 ranks the top-10 most discriminative features. Hashtag velocity (0.142) is the single strongest predictor, followed by retweet count (0.118) and sentiment polarity (0.097). TF-IDF terms contribute collectively but individually rank below engagement signals.

Fig. 3. Top-10 feature importances from Random Forest (normalised mean decrease in impurity).

D. Time-Series Trend Detection

Fig. 4 illustrates predicted trend probability versus actual trending status over 72 hours for #GlobalClimateSummit. The model surpasses the 0.70 confidence threshold approximately 4.2 hours before the topic officially trended.

Fig. 4. Predicted trend probability vs. actual trending status over 72 hours. Model alert precedes official trend onset by ~4 hours.

E. Cross-Validation Stability

Fig. 5 plots 5-fold CV accuracy for all three models. Random Forest yields $91.0 \pm 0.55\%$, confirming low variance and reliable generalisation. Table II provides per-fold breakdown.

Fig. 5. 5-fold cross-validation accuracy for all three classifiers.

Fold

1

2

3

Matched Source

Similarity 2%

Title:[How to calculate the F1 score and other custom metrics in PyTorch?](#)

Jul 23, 2025 · Calculating Model Metrics: Precision, Recall, F1-score, ROC AUC Set model to evaluation mode: `model.eval()` is used to set the model to evaluation mode. This disables any operations like dropout that are only meant to be applied during training. Iterate over test dataset: The code iterates over the test dataset...

<https://www.geeksforgeeks.org/deep-learning/how-to-calculate-the-f1-score-and-other-custom-metrics-in-pytorch>

Check By:  Dupli Checker